



Funciones básicas de GeoPandas

Primitivas gráficas con Shapely

Toda organización o empresa que opere en el mundo físico necesitará, en algún momento, visualizar **datos geoespaciales**. Desde la delimitación de áreas de trabajo hasta el diseño de rutas de evacuación, la ubicación de activos o la identificación de puntos clave, la geovisualización se convierte en **una herramienta esencial para que puedas tomar decisiones informadas**.

Iniciaremos la lección explorando la definición e importancia de los elementos gráficos fundamentales para representar datos espaciales: **puntos, líneas y polígonos**, para posteriormente aprender a manipularlos utilizando la librería **Shapely**, una herramienta que comúnmente se usa junto con GeoPandas para la visualización y análisis de datos geoespaciales.

Punto

Un punto representa una ubicación en un espacio de una, dos o tres dimensiones, ya sea sobre una recta, en un plano o en el espacio tridimensional. Sin embargo, se considera un elemento sin dimensiones, ya que no posee longitud, área ni volumen.

El punto es una **noción primitiva en geometría**, lo que significa que no puede definirse en función de otros elementos geométricos. Por el contrario, sirve como base para definir estructuras más complejas.

Clase Point

En Python, la librería Shapely te permitirá trabajar con puntos de forma sencilla y eficiente. En Shapely, un `punto(Point)` es un objeto que representa una ubicación en el espacio, pudiendo ser bidimensional (x, y) o tridimensional (x, y, z).

Para crear un punto a través de esta clase, basta con proporcionar las coordenadas correspondientes. Estas pueden ser valores enteros o decimales, según la precisión requerida.

A continuación, te mostramos un ejemplo de cómo definir un punto usando la biblioteca Shapely:

1. Antes de iniciar, debes importar de la librería `shapely.geometry` la clase `Point`.

```
from shapely.geometry import Point
```

2. Ahora vas a declarar algunos puntos, intenta usar valores enteros y flotantes.

```
punto1 = Point(2.2, 4.2)
punto2 = Point(7, -25)
punto3 = Point(9.26, -2.456)
punto_3d = Point(9.26, -2.456, 0.57)
```

3. Puedes visualizar la información de los objetos que acabas de declarar usando la función `print`.

```
print(punto1)
print(punto2)
print(punto3)
print(punto_3d)
```

4. Al imprimir la información con la función `print`, puedes visualizar el siguiente resultado:

```
POINT (2.2 4.2)
POINT (7 -25)
POINT (9.26 -2.456)
POINT Z (9.26 -2.456 0.57)
```

Observa que, para que la notación sea más clara, el método `print` agrega una **Z** para indicar que el último punto está definido en tres dimensiones.

Atributos y funciones

La clase `Point` en Shapely proporciona varios atributos y funciones predefinidas que puedes utilizar directamente, por ejemplo:

- **`geom_type`**: indica el tipo de geometría.
- **`Coords`**: permite acceder a las coordenadas del punto a través de un objeto `CoordinateSequence`.
- **`distance()`**: calcula la distancia entre dos puntos.

geom_type

```
▶ punto1.geom_type  
↳ 'Point'
```

Coords

Puedes convertir la secuencia de coordenadas a una **lista** para visualizarla mejor u obtener los valores de cada coordenada con los atributos **x**, **y**, **z**.

```
punto1.coords  
  
<shapely.coords.CoordinateSequence at 0x7d0fd5c976d0>
```

```
list(punto1.coords)  
  
[(2.2, 4.2)]
```

```
print(punto_3d.x)  
print(punto_3d.y)  
print(punto_3d.z)
```

```
9.26  
-2.456  
0.57
```

distance

```
punto1.distance(punto2)
```

```
29.591890781090687
```

Línea

La clase **Línea** (`Line`) se considera una **primitiva geométrica** que se define a partir de dos puntos y puede ser recta o curva. Una línea recta, comúnmente descrita como la distancia más corta entre dos puntos, puede existir tanto en un plano bidimensional (2D) como en el espacio tridimensional (3D).

LineString

En Python, la clase **LineString** del módulo **Shapely** permite representar secuencias de puntos conectados para formar líneas. Si se utilizan sólo dos puntos, se obtiene una línea recta; al agregar más puntos, se generan secuencias de segmentos de línea recta que pueden aproximar curvas.

En Python, la clase **LineString** del módulo **Shapely** permite representar secuencias de puntos conectados para formar líneas. Si se utilizan sólo dos puntos, se obtiene una línea recta; al agregar más puntos, se generan secuencias de segmentos de línea recta que pueden aproximar curvas.

Observa los siguientes pasos que te muestran cómo crear y manipular una **LineString** en Shapely:

1. Antes de iniciar, debes importar de la biblioteca **shapely.geometry** la clase `Point` y la clase `LineString`.

```
from shapely.geometry import Point, LineString
```

2. Para crear una **LineString** en Shapely, basta con instanciar el objeto y pasar como parámetro una lista de puntos:

```
linea = LineString([punto1, punto2, punto3])
linea2 = LineString([(2.2, 4.2), (7, -25), (9.26,
-2.456)])
linea
```



Figura 1. LineString.

3. Para visualizarla, escribe el nombre de la variable, o usa **print()** para obtener una descripción detallada de la **LineString**. Además, es posible comparar dos **LineString** utilizando el operador **==**.

```
print(linea)
linea == linea2
```

```
LINESTRING (2.2 4.2, 7 -25, 9.26 -2.456)
True
```

Atributos

Estos son algunos atributos de la clase `LineString` que te podrían ser de utilidad.

Atributo	Definición	Ejemplo
<code>coords</code>	Es posible obtener las coordenadas de una LineString mediante el atributo coords . Este devuelve un objeto CoordinateSequence , que puede convertirse en una lista para facilitar su visualización.	<pre>print(linea.coords) list(linea.coords)</pre> <pre><shapely.coords. CoordinateSequence object at 0x79f6efb690> [(2.2, 4.2), (7.0, -25.0, -2.456)]</pre>
<code>xy</code>	Este atributo permite obtener por separado todas las coordenadas x o todas las coordenadas y de la LineString .	<pre>xcoords = list(linea.xy[0]) print(xcoords) ycoords = list(linea.xy[1]) print(ycoords)</pre> <pre>[2.2, 7.0, 9.26] [4.2, -25.0, -2.456]</pre>
<code>length</code>	Este atributo devuelve la longitud de la LineString , calculada como la suma de las longitudes de todos sus segmentos.	<pre>longitud = linea.length print(longitud)</pre> <pre>52.24888828738</pre>

Atributo	Definición	Ejemplo
centroid	Este atributo devuelve un punto que representa el centroide de la LineString . El centroide es el punto de equilibrio geométrico de la figura, calculado a partir de la distribución de sus coordenadas.	<pre>print(linea.centroid)</pre> <pre>POINT (6.130734984394282 -11.843140517865203)</pre>

Tabla 1. Atributos de la clase `LineString`.

Polígono

La clase **Polígono** representa una **figura geométrica delimitada por un conjunto finito de segmentos de línea**, donde el punto final del último segmento coincide con el punto inicial del primero, formando un contorno cerrado.

Los **polígonos simples** son aquellos que no se autointersectan. Estos serán el tipo de polígonos de mayor interés en nuestro análisis.

Clase `Polygon`

En Python, un polígono se representa mediante un objeto **`Polygon`** en **Shapely**, el cual define un área cerrada delimitada por al menos tres coordenadas que conforman su perímetro. Opcionalmente, también es posible especificar una lista de huecos dentro del polígono.

A continuación, te mostramos cómo definir un polígono:

```
from shapely.geometry import Point, LineString, Polygon
poligono = Polygon([(2.2, 4.2), (7.2, -25.1), (-18, -12)])
poligono
```

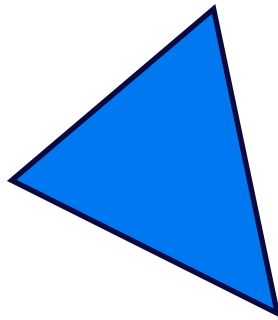


Figura 2. Polígono.

Ahora veamos cómo declarar polígonos con huecos.

Para ello, primero se define el contorno como una lista de pares ordenados, luego se especifica el hueco y, finalmente, se crea el polígono completo.

```
contorno = [(-180, 90), (-180, -90), (180, -90), (180, 90)]
hueco = [(0, 60), (-140, -60), (170, 0)]
poligono_con_hueco = Polygon(shell=contorno, holes=hueco)
poligono_con_hueco
```

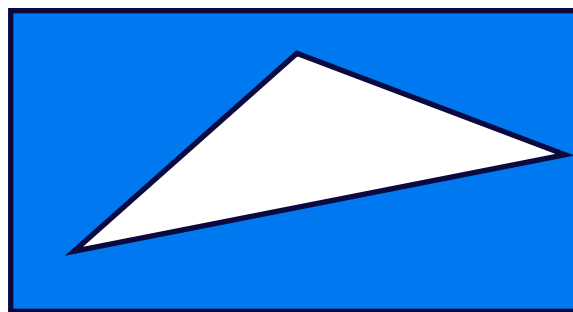


Figura 3. Polígono hueco.

Atributos

Al igual que los puntos y las líneas, los polígonos cuentan con varios atributos que pueden resultar útiles:

Atributo	Definición	Ejemplo
centroid	Regresa el centroide del polígono como un objeto de tipo Point .	<pre>print("El centroide del polígono es: ", poligono_con_hueco.centroid) print("El área del polígono es: ", poligono_con_hueco.area) print("El rectángulo mínimo que contiene al polígono es: ", poligono_con_hueco.bounds) print("El exterior del polígono es: ", poligono_con_hueco.exterior) print("El perímetro del polígono es: ", poligono_con_hueco.exterior.length)</pre>
area	Regresa el área del polígono.	
bounds	Regresa el polígono rectangular mínimo que encierra al polígono.	
exterior	Regresa un objeto LinearRing , que define el borde exterior del polígono. Un LinearRing es similar a una LineString , pero se diferencia en que es cerrada, lo que significa que el primer punto coincide con el último.	
length	Regresa el perímetro del polígono.	

El centroide del polígono es:
El área del polígono es:
El rectángulo mínimo que contiene al polígono es:
El exterior del polígono es:
El perímetro del polígono es:

POINT (-2.857142857142857 0)
50400.0
(-180.0, -90.0, 180.0, 90.0)
LINEARRING (-180 90, -180 -90, 180 -90, 180 90, -180 90)
1080.0

Tabla 2. Atributos de los polígonos.

En Python, dentro de la misma biblioteca **Shapely**, existen objetos adicionales que facilitan la creación y gestión de colecciones de puntos, líneas y polígonos:

- **MultiPoint**: clase en Shapely que representa una colección de puntos, estructurados como una lista de tuplas de coordenadas.
- **MultiLineString**: colección de líneas almacenada como una secuencia.
- **MultiPolygon**: colección de polígonos, representada como una lista de secuencias, que define tanto el contorno exterior como, opcionalmente, los huecos dentro de cada polígono.

Ahora que comprendes los conceptos de punto, línea y polígono, también es importante que identifiques qué utilidades le puedes dar a cada uno, por ejemplo:

- Los **puntos** son útiles para referenciar ubicaciones específicas en un plano o mapa.
- Las **líneas**, si se tienen las relaciones adecuadas, pueden representar rutas o desplazamientos de alguna entidad.
- Finalmente, los **polígonos** son ideales para delimitar áreas o regiones, como las ciudades de un estado o los estados dentro de un país.

Reflexiona sobre lo siguiente:

Imagina que hay un nuevo integrante en tu equipo de trabajo:

- Si necesita usar con frecuencia las impresoras, ¿cómo facilitarías su ubicación?
- Si debe llegar rápidamente a una impresora sin pasar por diferentes áreas, ¿cómo le indicarías la ruta más rápida?
- Si necesita orientarse en el espacio de trabajo, ¿cómo delimitarías las áreas?

GeoSeries y GeoDataFrame

Relación entre Shapely, Pandas y GeoPandas

Observa la siguiente imagen, donde podrás ver las dos estructuras de datos principales que utilizarás en GeoPandas: GeoSeries y GeoDataFrame, las cuales se heredan a su vez de Series y DataFrame, contenidas en Pandas, como se ilustra a continuación:

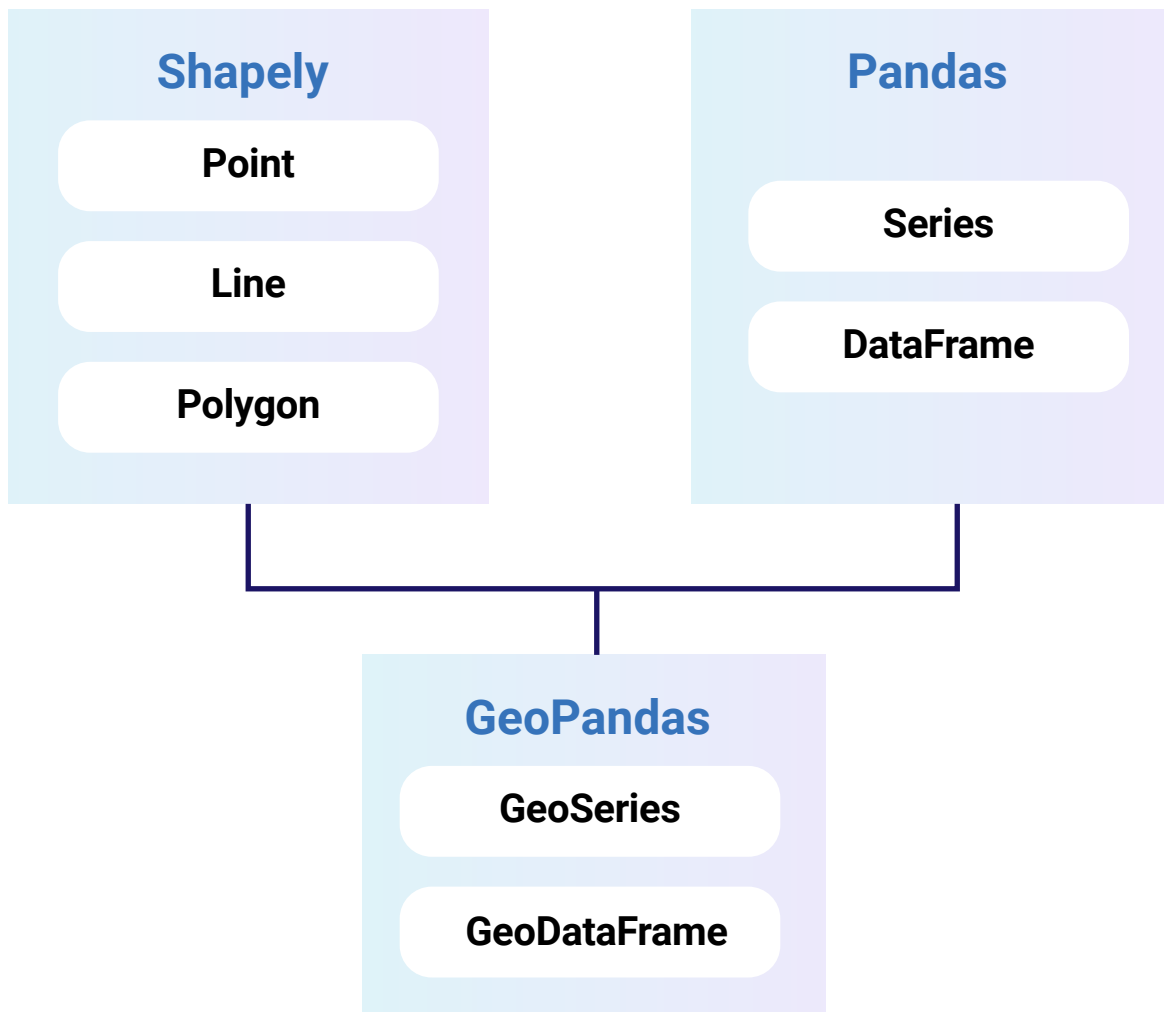


Figura 4. Relación entre Shapely, Pandas y GeoPandas.

GeoSeries

GeoSeries es una de las estructuras de datos más importantes de GeoPandas; sin embargo, para poder entenderla mejor se sugiere revisar la clase `Series` de la que **hereda**. La clase `Series`, es un arreglo unidimensional que tiene una etiqueta no necesariamente única y que puede contener elementos de cualquier tipo de datos (numéricos, *strings*, booleanos, listas, diccionarios, etc.).

En la **Programación Orientada a Objetos**, la herencia es un mecanismo que permite crear clases más específicas a partir de una clase más general, que a su vez es conocida como clase base o clase padre, y la clase que hereda, la más específica se conoce como clase hija o clase derivada.

GeoSeries también trabaja con un arreglo unidimensional, no obstante, la diferencia con la clase `Series` radica en que **cada entrada es un conjunto de figuras (*Shapes*) que corresponden a un sólo elemento**. De esta forma, una entrada de GeoSeries puede ser únicamente un polígono o varias formas que componen una ciudad, un estado, un país, un continente o incluso todo el mundo.

A continuación, se presentan algunos de los **atributos** más comunes de la clase GeoSeries:

Atributo	Descripción
area	Regresa una <code>Series</code> que contiene el área de cada geometría en la GeoSeries
bounds	Regresa un <code>DataFrame</code> con columnas <code>minx</code> , <code>miny</code> , <code>maxx</code> , <code>maxy</code> , que contienen los límites de cada geometría.

Atributo	Descripción
<code>total_bounds</code>	Regresa una tupla conteniendo <code>minx</code> , <code>miny</code> , <code>maxx</code> , <code>maxy</code> , con valores que representan los límites de las series completa.
<code>geom_type</code>	Regresa una <code>Series</code> de <i>strings</i> especificando el tipo de geometría de cada objeto.

Tabla 3. Atributos de GeoSeries.

Estos son sólo algunos de los atributos de la clase Geoseries. Para un listado completo, te sugerimos revisar la documentación de la clase GeoSeries que podrás encontrar en la sección de Material adicional, al final de esta lectura.

Por otro lado, estos son algunos de los **métodos** más utilizados de la clase GeoSeries:

Método	Descripción
<code>distance()</code>	Regresa las series con menor distancia entre una entrada y otra.
<code>centroid()</code>	Regresa una GeoSeries de centroides o centros de simetría de las figuras geométricas.
<code>representative_point()</code>	Regresa una GeoSeries de puntos que se garantiza que están dentro de cada figura geométrica, sin embargo, no regresa los centroides.
<code>plot()</code>	Grafica los elementos de la GeoSeries.

Método	Descripción
<code>geom_almost_equals()</code>	Identifica si la figura es casi idéntica a otra figura. Podría ser que sólo sean diferentes debido a problemas de precisión.
<code>cointains()</code>	Identifica si una figura está contenida dentro de otra.
<code>intersects()</code>	Identifica si una figura se intersecta con otra.

Tabla 4. Métodos de GeoSeries.

Estos son sólo algunos de los métodos de la clase Geoseries, para un listado completo, te sugerimos revisar la documentación de la clase GeoSeries que podrás encontrar en la sección de Material adicional, al final de esta lectura.

GeoDataFrame

Un GeoDataFrame es una estructura similar a una tabla que contiene GeoSeries. La característica que lo define es que contiene una columna con una GeoSeries denominada **geometría**. Cuando se aplica un método referente a datos espaciales sobre el GeoDataFrame, las operaciones se van a realizar sobre la columna geometría, la cual no necesariamente se debe llamar *geometry*, sino que puede tener cualquier otro nombre.

A pesar de que un GeoDataFrame puede contener más de una columna que contenga objetos que representan figuras geométricas, solamente una de ellas puede ser la geometría del DataFrame en un momento dado. El GeoDataFrame mantiene la información de cuál es la columna que identifica como su geometría a través de un atributo en el que guarda el nombre de esa columna, por lo que, **si se va a renombrar la columna que se considera su geometría, también se debe actualizar el atributo *geometry* del GeoDataFrame.**

Antes de terminar...

Ahora que ya conoces las dos estructuras de datos más importantes utilizadas en la librería GeoPandas, así como algunos de sus atributos y métodos más utilizados, piensa en los datos a los que puedes acceder en tu organización y reflexiona:

- ¿Es posible representar alguna parte de esa información utilizando los conceptos revisados, como puntos, líneas o polígonos, y guardar los datos en una GeoSeries o un GeoDataFrame?
- ¿Cuáles son las figuras geométricas que consideras más importantes para representar la información que puedas conseguir?
- ¿Existe algún tipo de información geoespacial que antes no sabías cómo representar, pero ahora puedes hacerlo utilizando primitivas gráficas como puntos, líneas o polígonos, y guardar en una GeoSeries o un GeoDataFrame?

Material adicional

Devakumar, K. (2020) *COVID-19 Dataset*. [Conjunto de datos del COVID-19]. Kaggle. Recuperado el 06 de marzo del 2025 de: <https://www.kaggle.com/imdevs-kp/corona-virus-report>

GeoPandas. (s.f). *Examples Gallery*. [Galería de ejemplos]. GeoPandas. Recuperado el 06 de marzo del 2025 de: <https://geopandas.org/en/stable/gallery/index.html>

GeoPandas. (s.f). *GeoSeries*. GeoPandas. Recuperado el 06 de marzo del 2025 de: <https://geopandas.org/en/stable/docs/reference/geoseries.html>

GeoPandas. (s.f). *Introduction to GeoPandas*. [Introducción a GeoPandas]. GeoPandas. Recuperado el 06 de marzo del 2025 de: https://geopandas.org/en/stable/getting_started/introduction.html

Nota: Los enlaces a las páginas web incluidos en este Material adicional fueron consultados y verificados por última vez el 06 de marzo del 2025 y su disponibilidad depende del autor.

Glosario

- **CSV (*Comma-Separated Values*):** Formato de archivo de texto plano que almacena datos en forma de tabla, donde los valores están separados por comas. Se usa comúnmente para almacenar grandes volúmenes de datos.
- **GeoDataFrame:** Estructura de datos en GeoPandas que extiende un DataFrame de pandas para manejar datos geoespaciales. Cada renglón representa una entidad geográfica y contiene una columna especial de geometría.
- **GeoPandas:** Biblioteca de Python que extiende pandas para trabajar con datos espaciales de forma sencilla e intuitiva, facilitando la manipulación y el análisis de datos geográficos.
- **GeoSeries:** Estructura de datos en GeoPandas que representa una serie de objetos geométricos, similar a una Serie de pandas pero con soporte para operaciones espaciales.
- **Line:** Primitiva geométrica en Shapely que representa una línea compuesta por dos o más puntos conectados en el espacio.
- **MultiLineString:** Conjunto de múltiples líneas no necesariamente conectadas entre sí.
- **MultiPoint:** Conjunto de puntos agrupados en una sola entidad.
- **MultiPolygon:** Conjunto de polígonos agrupados en una única entidad geoespacial.

- 
- **Point:** Primitiva geométrica que representa un punto en el espacio, definido por sus coordenadas (x, y).
 - **Polygon:** Primitiva geométrica que representa una superficie cerrada delimitada por una o más líneas conectadas que forman un contorno.
 - **XLS:** Formato de archivo utilizado por Microsoft Excel para almacenar datos en hojas de cálculo, comúnmente usado para estructurar y analizar información tabular.

Bibliografía

GeoPandas. (s.f). *GeoDataFrame*. GeoPandas. Recuperado el 06 de marzo del 2025 de: <https://geopandas.org/en/stable/docs/reference/geodataframe.html>

GeoPandas. (s.f). *GeoPandas 1.0.1*. GeoPandas. Recuperado el 06 de marzo del 2025 de: <https://geopandas.org/en/stable/>

GeoPandas. (s.f). *GeoSeries*. GeoPandas. Recuperado el 06 de marzo del 2025 de: <https://geopandas.org/en/stable/docs/reference/geoseries.html>

López , P. (29 de junio del 2020). *¿Qué es un archivo CSV?* Geeknetic. Recuperado el 06 de marzo del 2025 de: <https://www.geeknetic.es/Archivo-CSV/que-es-y-para-que-sirve>

Polo, C. (s.f.) Diferencia entre csv y xls. *Euroinnova*. Recuperado el 06 de marzo del 2025 de: <https://www.euroinnova.com/blog/diferencia-entre-csv-y-xls>

PyPi. (30 de enero del 2025). *Shapely 2.0.7*. PyPi. Recuperado el 06 de marzo del 2025 de: <https://pypi.org/project/Shapely/>

Shapely. (s.f). *shapely.LineString*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.LineString.html>

Shapely. (s.f). *shapely.MultiLineString*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.MultiLineString.html>

Shapely. (s.f). *shapely.MultiPoint*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.MultiPoint.html>

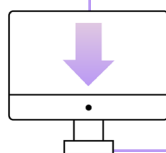
Shapely. (s.f). *shapely.MultiPolygon*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.MultiPolygon.html>

Shapely. (s.f). *shapely.Point*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.Point.html>

Shapely. (s.f). *shapely.Polygon*. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.Polygon.html>

Shapely. (31 de enero del 2025). *The Shapely User Manual*. [El Manual del Usuario de Shapely]. Shapely. Recuperado el 06 de marzo del 2025 de: <https://shapely.readthedocs.io/en/stable/manual.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 06 de marzo del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.